A-DAPT Blueprint Sprint 2

ACCEPTANCE TEST

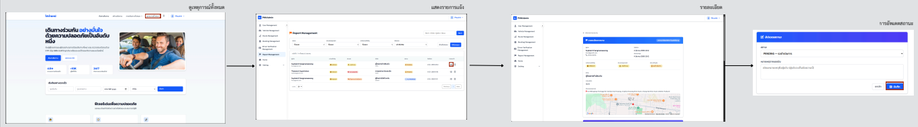
CUSTOMER VISIBLE

USER INTERFACE (UI)

User (คนขับ)

User (ผู้โดยสาร)

Admin



CUSTOMER INVISIBLE

BUSINESS LOGIC

การเข้าสู่ระบบ (Authentication Logic)
- ผู้ใช้ต้องมี Login ก่อนใช้งานระบบ
- ระบบตรวจสอบ Email + Password กับฐานข้อมูล
- ถ้าถูกต้อง → สร้าง Session / Token ไว้ที่เซิร์ฟเวอร์
- ระบบจะรู้ว่าเป็น
  - User ปกติ
  - หรือ Admin
- สิทธิ์การเข้าถึงแบ่งออกตามค่าอ้างอิงตาม Role

การจัดการรายงานโดย Admin (Admin Management Logic)
Admin เปิดหน้า /admin/reports
ระบบจะ:
ดึงรายงานทั้งหมด
พร้อมรองรับ:
- Filter ตาม Status
- Filter ตาม Incident Type
- Filter ตาม Priority
- Search keyword
- Pagination

เพื่อให้จัดการข้อมูลจำนวนมากได้

การสร้างรายงานเหตุการณ์ (Create Incident Report)
เมื่อ User แจ้งเหตุ ระบบทำงานตามลำดับนี้:
Step 1 — รับข้อมูลจากหน้าจอ
User ส่งมาพร้อม:
- ประเภทเหตุการณ์ (Incident Type)
- ชื่อเรื่อง
- รายละเอียด
- ระดับความสำคัญ
- ตำแหน่งปักหมุด (ถ้ามี)
- ผลกระทบ (ถ้ามี)
- ถ้าข้อมูลไม่ครบ ระบบจะไม่ให้ส่ง

Admin ตรวจสอบรายละเอียด
เมื่อเหตุการณ์ถูก
ระบบโหลด:
- ข้อมูลผู้แจ้ง
- รายละเอียดเหตุการณ์
- รูปภาพ (ถ้ามีการแนบ)
- และที่ตำแหน่งเดิมเหตุ

ใช้ location ที่ user ปักหมุดตามสดของระบบ read-only

การดำเนินผลตามกระบวนการ (Workflow Processing)
Admin สามารถเปลี่ยนสถานะได้ 4 แบบ:
Status ตามกระบวนการ
PENDING
รอดำเนินการ
IN_PROGRESS
กำลังการแก้ไข
RESOLVED
แก้ไขแล้ว
REJECTED
ปฏิเสธ
เมื่อ Admin กดบันทึก:
- ระบบ Update สถานะใน Database
- เพิ่ม Admin Note
- User จะเห็นสถานะใหม่ทันที

ทำให้เกิด Workflow การติดตามเป็นลูก

จัดการไฟล์รูปภาพ
ถ้าผู้โดยสารมีรูปแนบ:
- ระบบ Upload ไปเก็บที่ Cloudinary
- เก็บผลลาย URL ของรูปใน Database
- ไม่เก็บไฟล์จริงที่เซิร์ฟเวอร์ Server แต่เก็บไว้ที่ Cloud ที่ปลอดภัยกว่า

Step 2 — Validate ข้อมูลก่อนบันทึก
Backend จะตรวจสอบ:
- ความยาวข้อความไม่เกินที่กำหนด
- ชื่อ location (lat,lng) ถูกรูปแบบ
- ไฟล์รูปไม่เกิน 5MB
- ประเภทเหตุการณ์ต้องอยู่ใน ENUM ที่กำหนด

ENUM ที่มี:
- DRIVER, PASSENGER, ROUTE, BOOKING, SYSTEM, OTHER

บันทึกข้อมูลลง Database
Backend จะสร้าง Record ใหม่ในตาราง Report โดยมี:
- incidentType
- priority
- description
- location (lat,lng,address)
- imageUrl
- status = PENDING (ค่าเริ่มต้น)

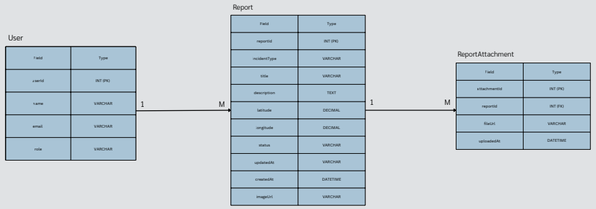
รายงานใหม่จุดนี้จะขึ้นทันทีในหน้าจอ "รอดำเนินการ"

การแสดงรายงานของผู้ใช้ (User View Logic)
เมื่อ User เปิดหน้า My Reports
ระบบจะ:
- Query รายงานเฉพาะของ user คนนั้น
- แสดง:
  - ประเภทเหตุการณ์
  - ระดับความสำคัญ
  - สถานะ
  - หมายเหตุจาก Admin
  - รูปภาพ + แผนที่

User เห็นเฉพาะของตัวเอง ไม่เห็นของคนอื่น

การลบรายงาน (Delete Logic)
ถ้า Admin ลบรายงาน
- ระบบจะลบ Record จาก Database ถาวร
- รายงานจะไม่สามารถกู้คืนได้

DATABASE



SUPPORT

Authentication & Security
- ระบบมีการ Login ก่อนใช้งาน (Authentication Required)
- ใช้การตรวจสอบตัวตนผ่าน Email / Password
- มีการแยกสิทธิ์ผู้ใช้:
  - User → แจ้งเหตุ / ดูรายงานตัวเอง
  - Admin → จัดการรายงานทั้งหมด
- ป้องกันการเข้าถึง API โดยไม่ได้ Login
- ใช้ Token / Session เพื่อรักษาสถานะการใช้งาน

ทำให้ข้อมูลเหตุการณ์ไม่ถูกเข้าถึงจากบุคคลภายนอก

ORM (Object-Relational Mapping)
- ใช้ Prisma ORM จัดการ Database
- Backend ไม่ต้องเขียน SQL ตรง ๆ
- แปลงคำสั่งจาก Code → Query Database อัตโนมัติ
- ช่วยให้:
  - ลด Bug จาก SQL
  - โครงสร้างข้อมูลชัดเจน
  - แก้ไข Schema ได้ง่าย
- ใช้ Prisma Model สำหรับ Report, User, Location ฯลฯ

ทำให้ระบบพัฒนาเร็วและ Maintain ง่าย

RESTful API
- Backend ให้บริการผ่าน REST API
- Frontend เรียกใช้งานผ่าน HTTP เช่น:
  - POST /reports → สร้าง Incident
  - GET /reports → ดึงข้อมูลรายงาน
  - PATCH /reports/:id → อัปเดตสถานะ
- API เป็นตัวกลางระหว่าง:
- Frontend (UI) ⇄ Backend Logic ⇄ Database

ทำให้ระบบแยกส่วน (Decoupled Architecture)

File Storage (Cloudinary)
- ระบบไม่เก็บรูปบน Server
- Upload รูปไปที่ Cloudinary
- Database เก็บแค่ลาย URL

ผลการ Server และเพิ่มความเร็วในการโหลด
รองรับการแนบหลักฐานเหตุการณ์ได้จริง

Map & Location Service
- ใช้รายบพิกัด Latitude / Longitude
- ผู้ใช้ต้องปักหมุดก่อนส่งรายงาน
- Backend บันทึกตำแหน่งเพื่อให้ Admin ตรวจสอบบนแผนที่ได้

ทำให้ Incident มีตำแหน่งจริง ตรวจสอบได้